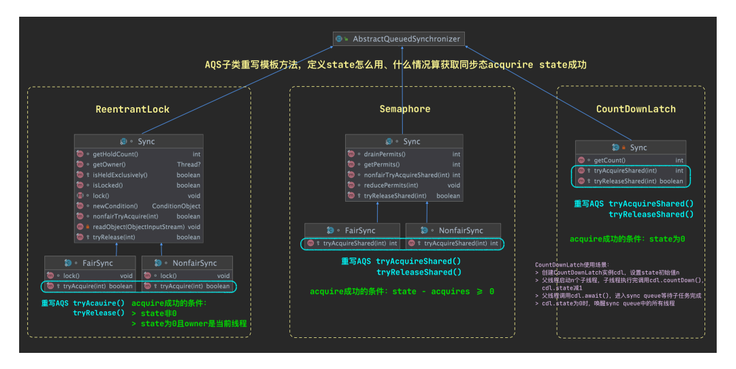
```
ReentrantLock.lock() → FairSync.lock() → AQS.acquire() → FairSync.tryAcquire() → AQS.addWaiter()  → AQS.acquireQueued()
ReentrantLock.unlock() → AQS.release() → FairSync.tryRelease() → AQS.unparkSuccessor()
CountDownLatch.countDown() → AQS.releaseShared() → sync.tryReleaseShared() → AQS.doReleaseShared()
CountDownLatch.await() → AQS.acquireSharedInterruptibly() → sync.tryAcqureShared() → AQS.doAcquireSharedInterruptibly()
Semaphore.acquire() → AQS.acquireSharedInterruptibly() → Semaphore.FaireSync.tryAcqureShared() → AQS.doAcquireSharedInterruptibly()
```

```
//1.
AQS.acquire(int arg) {
  if (!tryAcquire(arg) && acquireQueued(addWaiter(Node.EXCLUSIVE), arg)) selfInterrupt();
}

//2.
AQS.release(int arg) {
  if (tryRelease(arg)) {
    Node h = head;
    if (h ≠ null && h.waitStatus ≠ 0) unparkSuccessor(h);
    return true;
  }
  return false;
}

//3.
AQS.acquireSharedInterruptibly(int arg) {
  if (tryAcquireShared(arg) < 0) doAcquireSharedInterruptibly(arg);
}

//4.
AQS.releaseShared(int arg) {
  if (tryReleaseShared(arg)) {
    doReleaseShared();
    return true;
  }
  return false;
}
```


AbstractQueuedSynchronizer
AQS子类重写模板方法，定义state怎么用、什么情况算获取同步态acqurire state成功
ReentrantLock
Sync
getHoldCount() int
getOwner() Thread?
isHeldExclusively() boolean
isLocked() boolean
lock() void
newCondition() ConditionObject
nonfairTryAcquire(int) boolean
readObject(ObjectInputStream) void
tryRelease(int) boolean
FairSync
lock() void
tryAcquire(int) boolean
NonfairSync
lock() void
tryAcquire(int) boolean
重写AQS tryAcauire()
tryRelease()
acquire成功的条件:
> state非0
> state为0且owner是当前线程
Semaphore
Sync
drainPermits() int
getPermits() int
nonfairTryAcquireShared(int) int
reducePermits(int) void
tryReleaseShared(int) boolean
FairSync
tryAcquireShared(int) int
NonfairSync
tryAcquireShared(int) int
重写AQS tryAcquireShared()
tryReleaseShared()
acquire成功的条件：state - acquires ≥ 0
CountDownLatch
Sync
getCount() int
tryAcquireShared(int) int
tryReleaseShared(int) boolean
重写AQS tryAcquireShared()
tryReleaseShared()
acquire成功的条件：state为0

CountDownLatch使用场景：
> 创建CountDownLatch实例cdl，设置state初始值n
> 父线程启动n个子线程，子线程执行完调用cdl.countDown()，cdl.state减1
> 父线程调用cdl.await()，进入sync queue等待子任务完成
> cdl.state为0时，唤醒sync queue中的所有线程

Synchronizer	State	When State Doesn’t Permit Acquire
ReentrantLock	0：可用 1：不可用 大于1：线程重入次数	state非0 & 当前线程不是owner时
ReentrantReadWriteLock	高16位：读锁（共享） 低16位：写锁（排他，同ReentrantLock）	读锁： 写锁：
CountDownLatch	0：可用 非0：不可用	state非0时
Semaphore	大于0：可用 小于等于0：不可用	state - acquires < 0时